Treatment duration

Henrik Renlund

April 24, 2026

Contents

1 Setting

This vignette describe functions that deal with the calculation of treatment durations. The setting is that we get information about the dispensation of drugs from we we wish to extract time periods where the subject is (plausibly) on or off treatment.

Sometimes it might be plausible to assume that a dispensation is meant to last a fixed amount of time so that the functions in Section 2 are enough.

If the drug is taken as pill, with a known consumption rate, then the dispensation size can guide us to the treatment duration - see Section 3.

2 Fixed duration

Information about treatment initiation with fixed duration is handled with `fixed_treatment`. The information should be contained in a data frame with the following variables

- `id` subject id
- `t` the time variable
- `state` treatment
- `run` duration (can also be given as a number to `fixed_treatment` if it is not present in the data)

Note: the variables need not have these names, you can specify their actual names in `fixed_treatment`.

```
d <- rowwiseDT(
    id=, t=, state=, run=,
    1,    1,    "A",    5,
    1,   10,    "B",   15,
    1,   20,    "C",   50,
    1,   21,    "C",    9
)
(ft <- fixed_treatment(data = d, null.state = "*"))

##       id     t  state
##    <num> <num> <char>
## 1:     1     1      A
## 2:     1     6      *
## 3:     1    10      B
## 4:     1    20      C
## 5:     1    30      *
```

Note that the treatment (A) initiated on day 1 lasts 5 days, so at day 6 the treatment status reverts to the null state (since no other treatment is initiated). The treatment (B) initiated on day 10 could last 15 days but is interrupted by treatment (C) on day 20. On day 20 C is initated to last a full 50 days but is updated on day 21 to only last 9 more days and thus treatment is over on day 30.

This example is meant to illustrate "competing drugs" for the same treatment, e.g. different oral anticoagulants for atrial fibrillation. If one is considering drugs that can be taken simultaneously, one would simply apply `fixed_treatment` several time on different drug data.

Getting treatment durations is often useful in connection with time-dependent covariates which can be created with the `survival::tmerge` function.

```
b <- data.table(id=1, tstart=0, tstop=50)
T0 <- tmerge(b, b, id = id, tstart = tstart, tstop = tstop)
tmerge(T0, ft, id = id, trt = tdc(t, state, init = "*"))

##   id tstart tstop trt
## 1  1      0     1   *
## 2  1      1     6   A
## 3  1      6    10   *
## 4  1     10    20   B
## 5  1     20    30   C
## 6  1     30    50   *
```

When there is a single treatment one can use the wrapper `onoff_treatment`, for which one need *not* provide a `state` variable.

```
d <- rowwiseDT(
    id=, t=,  run=,
    1,    1,     5,
    1,   10,    99,
    1,   20,    10
)
onoff_treatment(data = d)

##       id     t state
##    <num> <num> <int>
## 1:     1     1     1
## 2:     1     6     0
## 3:     1    10     1
## 4:     1    30     0
```

3 Pill-based duration

Information about treatment initiation with pill-based duration is handled with `pill_treatment`. The information should be contained in a data frame with the following variables

- `id` subject id
- `t` the time variable
- `state` treatment state
- `pills` number of pills provided at this time
- `usage` daily number of pills to consume (at this time). Can also be given to `pill_treatment` as an integer.
- `capacity` the maximum number of pills that can be kept in storage (at this time, applied *after* the daily dose has been consumed). Can also be given to `pill_treatment` as an integer.

Note:

- the variables need not have these names, you can specify their actual names in `fixed_treatment`.
- *Pills are stored.* Unlike `fixed_treatment` where a new duration directive for the same treatment kills the previous duration, an addition of pills will just add to the stockpile of pills. *However,* this is only true if the pills are for the same treatment.

```
d <- rowwiseDT(
    id=, t=, state=, pills=, usage=, capacity=,
    1,    1,    "A",     10,      2,       Inf,
    1,   10,    "A",   1000,      1,         5,
    1,   20,    "A",     10,      1,       Inf,
    1,   25,    "A",     10,      1,       Inf,
    1,  100,    "A",     50,      1,       Inf,
    1,  110,    "B",     50,      1,       Inf
)
pill_treatment(data = d)

##       id     t  state
##    <num> <num> <char>
## 1:     1     1      A
## 2:     1     6
## 3:     1    10      A
## 4:     1    16
## 5:     1    20      A
## 6:     1    40
## 7:     1   100      A
## 8:     1   110      B
## 9:     1   160
```

Note: treatment A on day 1 is initiated with 10 pills but a daily usage of 2 so only lasts 5 days. Treatment A re-initiated on day 10 gets 1000 pills but the stock-piling capacity is only 5, which takes effect after the daily dose is consumed making the duration of that treatment initiation only 6 days. The re-initiation on day 20 is prolonged by the pills received on day 25. The re-initiation of A on day 100 is terminated (and the stock-pile of A-pills is nullified) on day 110 by a new treatment B.

4 Related topics

4.1 Treatment burden

Here we want to calculates some measure of how much treatment (*a single treatment*) a subject is exposed to. It is not obvious how to create such a measure in a meaningful way. The function `pill_burden` is an attempt to do so.

To see what is calculated, lets look at (part of) the rather large return data frame we get from a call.

```
d <- rowwiseDT(
    id=, t=,  pills=,
    1,    1,      16
)
pb <- pill_burden(data = d, usage = 2, capacity = Inf, window = 4,
                  burden = "treatment", dep.rate = 1)
pb[, .(id, t, store.begin, store.end, use, trt, stat, cumstat)]

##        id     t store.begin store.end   use   trt  stat cumstat
##     <num> <num>       <num>     <num> <num> <int> <num>   <num>
##  1:     1     1          16        14     2     1     1       1
##  2:     1     2          14        12     2     1     1       2
##  3:     1     3          12        10     2     1     1       3
##  4:     1     4          10         8     2     1     1       4
##  5:     1     5           8         6     2     1     1       4
##  6:     1     6           6         4     2     1     1       4
##  7:     1     7           4         2     2     1     1       4
##  8:     1     8           2         0     2     1     1       4
##  9:     1     9           0         0     0     0    -1       2
## 10:     1    10           0         0     0     0    -1       0
```

Here

- `use` shows pill consumption
- `trt` shows treatment status (0/1)
- `stat` is what is to be summed, in this case - since we called the function with `burden = "treatment"` - it is +1 for treatment and `-dep.rate` (-1 in this example) for non-treatment.
- `cumstat` is the cumulative summation of `stat` within a window of length `window` (4 in this example), bounded to never drop below zero.

It is the `cumstat` column we are interested in.
If we repeat the example but change the `burden` argument, we instead get

```
pill_burden(data = d, usage = 2, capacity = Inf, window = 4,
            burden = "pill", dep.rate = 1)[
  , .(id, t, store.begin, store.end, use, trt, stat, cumstat)]
```

```
##        id     t store.begin store.end   use   trt  stat cumstat
##     <num> <num>       <num>     <num> <num> <int> <num>   <num>
##  1:     1     1          16        14     2     1     2       2
##  2:     1     2          14        12     2     1     2       4
##  3:     1     3          12        10     2     1     2       6
##  4:     1     4          10         8     2     1     2       8
##  5:     1     5           8         6     2     1     2       8
##  6:     1     6           6         4     2     1     2       8
##  7:     1     7           4         2     2     1     2       8
##  8:     1     8           2         0     2     1     2       8
##  9:     1     9           0         0     0     0    -1       5
## 10:     1    10           0         0     0     0    -1       2
## 11:     1    11           0         0     0     0    -1       0
```

The difference here is how `stat` is calculated, now it is on `use` instead of `trt`.

As is clear, this function generates a looong output and perhaps with too fine a detail. If a less fine `cumstat` can do, this can be achieved with using the `breaks` argument.

```
pill_burden(data = d, usage = 2, capacity = Inf, window = 4,
            burden = "pill", dep.rate = 1,
            breaks = c(-1,0,5,10),
            break.labels = c("no burden",
                             "1-5 pill burden",
                             "6-10 pill burden"),
            simplify = TRUE)
```

```
##       id     t       cumstatcat
##    <num> <num>           <fctr>
## 1:     1     1  1-5 pill burden
## 2:     1     3 6-10 pill burden
## 3:     1     9  1-5 pill burden
## 4:     1    11        no burden
```